

pp reference manual

a content-addressed, capability-scoped Lisp

Contents

1	Introduction	3
1.1	How to read this manual	4
2	The language in brief	4
2.1	Values	4
2.2	Bindings	4
2.3	Functions	4
2.4	Types	5
2.5	Laziness	5
2.6	Effects and handlers	6
2.7	Config	6
3	Nodes and content-addressing	7
3.1	The node key	7
3.2	Caching across runs	8
3.3	Pure compute needs no capability	9
4	The store and verifying traces	9
4.1	The store	9
4.2	A hit is a verified trace, not a timestamp	10
4.3	pp why	11
4.4	Auditing the cache	11
5	Foreign execution	12
5.1	Policy belongs to libraries and providers	12
6	Capabilities and secrets	13
6.1	Capabilities enter only at the root	13
6.2	User code narrows, never widens	14
6.3	Authority gates the cache, transitively	15
6.4	Secrets: sealed cells	15
7	Modules and islands	17
7.1	Modules	17
7.2	Loading from files	17
7.3	Islands	18
8	Domains and the reconciler	21
8.1	A domain is observe / diff / apply	21
8.2	Reconciling a filesystem	21
8.3	Watching, and other domains	23
8.4	Fenced effects	23
9	Distribution	23
9.1	Identity is location-independent	23
9.2	Transport boundary	26
9.3	Host-qualified identity	26
9.4	Trace merge, audit, and growth	26
10	The evaluator	26

10.1	The expected-output suite	26
10.2	Kernel properties	27
11	Language reference	27
11.1	Value types	27
11.2	Arithmetic and comparison	28
11.3	Booleans and conditionals	29
11.4	Bindings	29
11.5	Functions	30
11.6	Nodes	30
11.7	Sequencing	31
11.8	Type annotations	31
11.9	Quotation	32
11.10	Laziness	32
11.11	Effects and config	33
11.12	Assertions	34
11.13	Macros	34
11.14	Modules	35
11.15	Desired state	35
11.16	Lists	35
11.17	List and vector literals	37
11.18	Maps	38
11.19	Index access	39
11.20	Map update	40
11.21	Strings	40
11.21.1	Interpolation (f-strings)	42
11.22	Type predicates	43
11.23	Pattern matching	43
11.24	Spread	44
11.25	Observation sigils	45
11.26	Error propagation: try	46
11.27	Collecting results	46
12	Command-line reference	47
12.1	Running programs	47
12.2	Execution modes	47
12.3	Capabilities and grants	48
12.4	The store and auditing	48
12.5	Reconcile, watch, and supervise	48
12.6	Islands	49
12.7	Scheduling and distribution	49
12.7.1	Internal transport (not for hand use)	50
12.8	Observation pinning	50
13	Builtins and the standard library	51
13.1	Dune adapter	51
13.2	Core builtins	51
13.2.1	Arithmetic and comparison	51
13.2.2	Lists and pairs	51
13.2.3	Vectors, maps, and sets	51
13.2.4	Type predicates	52
13.2.5	Strings and numbers	52

13.2.6	I/O and control	52
13.2.7	Filesystem observation	53
13.2.8	Hashing and the content store	53
13.2.9	Capabilities	53
13.2.10	Effects, domains, and metaprogramming	53
13.3	Lists	54
13.4	Maps	54
13.5	Strings	54
13.6	Domains	55
13.6.1	Filesystem policy internals (domain-fs.pp)	55
13.6.2	Process policy internals (domain-proc.pp)	55

1 Introduction

pp is a content-addressed, capability-scoped Lisp: one evaluation model for work usually split among build systems, supervisors, and orchestrators — each managing computations, dependencies, reuse, and effects. pp supplies that substrate; toolchains and domain policy stay ordinary libraries and narrow trusted providers.

Two ideas carry the design. Every value has a content hash, so two computations with the same code and inputs are the same computation. Side effects require capabilities: unforgeable tokens granted only at the command line. Everything else follows.

The fastest way to learn pp is to read it running: every example is a real program run by the manual build; the output shown is what it printed. The smallest example:

hello.pp

```
# Every value has a content hash; arithmetic and comparison are variadic.
print(+(1, 2, 3))
print(if 5 > 0 { "positive" } else { "negative" })
print(6 * 7)
```

```
$ pp hello.pp
```

```
6
"positive"
42
```

The output appears below the command because the manual ran that file. If an example ever stopped producing the output shown, the build for this page would have failed.

That extends to mistakes. pp checks optional type annotations when a body runs; feed one the wrong type and it says so, naming the value:

type-error.pp

```
# Type annotations are checked when the body runs. Passing a string where
# an int is expected is caught — the manual shows the real error.
def increment(n: int): int { n + 1 }
increment("oops")
```

```
$ pp type-error.pp
```

```
pp: error: type mismatch: expected int, got "oops" at type-error.pp:3
```

Errors throughout this manual are provoked and shown, not described.

1.1 How to read this manual

The chapters move from the surface inward: values, bindings, functions, laziness, then the effect-and-capability system, then the build substrate — nodes and the store, foreign execution, modules and islands, domains and the reconciler, distribution. Each construct gets a sentence of motivation, then a run.

2 The language in brief

Enough pp to read the rest of the manual. pp is a Lisp-1: functions and variables share one namespace; evaluation is inner-first and strict. The language reference appendix has every form in full.

2.1 Values

pp has the atoms you expect: integers and floats, strings, the booleans `true` and `false`, `nil`, keywords like `:host`, and quoted symbols like `quote { name }`. Arithmetic and comparison are variadic. `if` is an expression: it returns a value.

hello.pp

```
# Every value has a content hash; arithmetic and comparison are variadic.
print(+(1, 2, 3))
print(if 5 > 0 { "positive" } else { "negative" })
print(6 * 7)
```

```
$ pp hello.pp
```

```
6
"positive"
42
```

2.2 Bindings

`let` introduces local bindings. They are mutual: every binding sees every other, whatever the order. For top-down behaviour use `let*`.

lang-bindings.pp

```
# Bindings in a `let` are mutual: every binding sees every other regardless
# of the order they are written. One flat `let` with multiple bindings is
# the idiomatic style – no nested single-binding ladders.
```

```
print(let (y = x + 1, x = 1) { y })
```

```
# `let*` is the sequential form, where each binding sees only the ones above.
```

```
print(let* (x = 1, x = x + 1) { x })
```

```
$ pp lang-bindings.pp
```

```
2
2
```

Both bind names to right-hand-side values. `def` does the same at top level, and also defines functions.

2.3 Functions

A definition and a function value are the same idea written two ways: `def` names one, `fn` produces one anonymously. No loop keyword: recursion is the loop, and tail calls do not grow the stack.

lang-functions.pp

```
# `def` names a function; the last expression is its result.
# Functions are values – `fn` makes one without naming it.
def square(x) { x * x }
print(square(7))

# Inline function as a value.
print((fn(x) { x * x })(7))

# Recursion is the only loop. Flat `else if` chains keep branching clean.
def factorial(n) {
  if n = 0 {
    1
  } else {
    n * factorial(n - 1)
  }
}

print(factorial(5))

$ pp lang-functions.pp
```

```
49
49
120
```

2.4 Types

Type annotations are optional, checked when a function's body runs. A mismatch names the value and location:

type-error.pp

```
# Type annotations are checked when the body runs. Passing a string where
# an int is expected is caught – the manual shows the real error.
def increment(n: int): int { n + 1 }
increment("oops")

$ pp type-error.pp
```

```
pp: error: type mismatch: expected int, got "oops" at type-error.pp:3
```

2.5 Laziness

delay builds a thunk: a computation that has not run yet. force runs it and memoizes the result. The work happens at most once, however many times you force it.

lang-laziness.pp

```
# `delay` builds a thunk that has not run yet; `force` runs it, and the
# result is memoized – the work happens at most once.
let (t = delay(do {
  print("working")
  100 * 200
})) {
  print(force(t))
  print(force(t))
}

$ pp lang-laziness.pp
```

```
"working"
20000
20000
```

Two identical thunks with the same inputs and environment are the same thunk: computed once, shared everywhere. Wrapping a computation in node (next chapter) extends that sharing across runs of the program.

2.6 Effects and handlers

Side effects go through `perform`, which dispatches to the nearest enclosing handler. Handling an effect needs no capability; only effects that touch the world fall under the authority model (capabilities chapter).

lang-effects.pp

```
# A side effect is a `perform` that dispatches to the nearest handler.
# This needs no capability: the authority model governs effects that touch
# the world, not ones you handle yourself. The `handlers:` clause takes a
# map-valued block – `{ :name -> fn, ... }`.
with { handlers: { :ask -> fn(q) { 42 } } } {
  print(perform ask("the answer?"))
}

$ pp lang-effects.pp
```

```
42
```

2.7 Config

Config is ambient, dynamically scoped data any code in extent can read — deliberately not a capability. Config is information; capabilities are authority.

lang-config.pp

```
# Config is ambient, dynamically-scoped data – distinct from capabilities,  
# which are authority. A binding is visible to everything called within.  
# $config(key) is the idiomatic observation sigil for config reads.  
print(with-config({:host -> "db1"}) { $config(:host) })  
  
$ pp lang-config.pp
```

```
"db1"
```

That is the surface. Everything from here on is about what happens when you wrap a computation in node and give pp the authority to act on the world.

3 Nodes and content-addressing

The previous chapter ended on delay and force: a thunk runs at most once, and identical thunks in one run are the same thunk. node extends that past the edge of a single run: wrap a computation in node and its result is content-addressed into a store on disk (~/.pp/store). The next computation asking for the same thing — later in this run, in another process, or on another machine sharing the store — finds the answer already there.

3.1 The node key

What makes two computations “the same thing” is a hash. A node’s key is

$H(\text{code} \parallel \text{free-var value-hashes})$

— the hash of the node’s code plus the values of the free variables it references (LAW 20). Nothing else: no ambient capabilities, config, handler stack, or rest of the environment. Identical code fed identical inputs gives an identical key; same key, same node.

Write the same node twice and pp treats them as one. The first force runs the body and stores the result. The second finds that result by key. A store hit does not replay whatever the body printed, so the side effect happens exactly once:

node-identity.pp

```
# A node is identified by its code and its inputs (LAW 20). Two nodes with  
# identical code and free-variable values are the SAME node: the first force  
# runs the body and stores the result; the second finds it already in the  
# store. A store hit does not replay the body's output (LAW 17), so  
# "compiling" prints exactly once.  
let first = node { print("compiling"); 6 * 7 }  
let second = node { print("compiling"); 6 * 7 }  
print(first)  
print(second)  
  
$ pp node-identity.pp
```

```
"compiling"  
42  
42
```

The key folds in free-variable values, not how they were computed. x and y below are built differently but are the same value, so the two nodes are one node and the body runs once. A different input means a different key, and a run:

node-freevars.pp

```
# The node key folds in the VALUES of the free variables the body references,  
# not the code that produced them (LAW 20). `x` and `y` are computed  
# differently but are the same value, so `squared(x)` and `squared(y)` are  
# the same node – the body runs once. A different value produces a different  
# key, so `squared(9)` runs separately.
```

```
def squared(n) { node { print("squaring"); n * n } }  
let x = 5  
let y = 2 + 3  
print(squared(x))  
print(squared(y))  
print(squared(9))
```

```
$ pp node-freevars.pp
```

```
"squaring"  
25  
25  
"squaring"  
81
```

Hence excluding the whole environment: rebind an unrelated global and nodes keyed on it are untouched; change a value a node actually reads and only that node re-keys.

3.2 Caching across runs

The store is on disk, so node results outlive the process. A delay memoizes within one run; a node memoizes across runs:

node-reuse.sh

```
#!/bin/sh
# A node's result is content-addressed in ~/.pp/store, so it survives the
# process that produced it. A second, separate run of the same program finds
# the result already there and does not re-run the body. A throwaway store (a
# temp HOME) keeps this hermetic.
export HOME=$(mktemp -d)
cd "$HOME"

cat > build.pp <<'PP'
def compile() { node { print("compiling greeter.o"); 6 * 7 } }
print(compile())
PP

echo '$ pp build.pp      # first process: the node body runs'
"$PP" build.pp

echo
echo '$ pp build.pp      # second process: served from the store, body skipped'
"$PP" build.pp

rm -rf "$HOME"
```

```
$ pp build.pp      # first process: the node body runs
"compiling greeter.o"
42

$ pp build.pp      # second process: served from the store, body skipped
42
```

The body prints on the first run, is silent on the second; the second process never entered the body. The force collapse made durable: equal computations run once, however many processes ask.

3.3 Pure compute needs no capability

None of these examples were granted anything — `node-identity` and `node-freevars` run with bare `pp file.pp`, since arithmetic touches nothing. A computation that only computes needs no capability; caching it needs none. The moment a node reads the world, the cache must account for what it saw: files, config, handlers, probes, and child results become trace cells; ambient subprocesses are rejected. Traces and foreign execution follow.

4 The store and verifying traces

A node's key decides identity: which computation this is. It does not decide whether a cached result is still good — `pp` answers that in a way that never trusts a clock.

4.1 The store

Everything a node produces lives under `~/.pp/store`, in three content-addressed parts:

store-layout.sh

```
#!/bin/sh
# The store has three content-addressed parts. A temp HOME keeps it throwaway.
export HOME=$(mktemp -d)
cd "$HOME"

cat > build.pp <<'PP'
let artifact = node { blob(slurp("input.txt")) }
print(artifact)

PP
echo hello > input.txt

echo '$ pp build.pp --grant fs:$HOME:ro'
"$PP" build.pp --grant "fs:$HOME:ro"

echo
echo '$ ls ~/.pp/store'
ls "$HOME/.pp/store"

rm -rf "$HOME"
```

```
$ pp build.pp --grant fs:$HOME:ro
"5891b5b522d5df086d0ffb110fbd9d21bb4fc7163af34d08286a2e846f6be03"

$ ls ~/.pp/store
VERSION
blobs
locks
objects
traces
```

- objects/ — result values, each file named by the hash of the value it holds.
- traces/ — one file per node key, holding the set of traces recorded for that node (see below).
- blobs/ — raw bytes ingested by blob(...), named by their hash. The program above turned a file's contents into a blob identity, and the bytes landed here.

(VERSION stamps the store format and locks/ guards concurrent writers. Neither is content you address directly.)

4.2 A hit is a verified trace, not a timestamp

When a node runs, pp records every observation of the world as a (cell, hash) pair: each file, config value, subprocess tool. That record is a verifying trace: not “this ran at 3pm” but “this is what the node saw.”

On the next force pp re-observes every cell in the trace and serves the stored result only if every hash still matches; no modification times. One key holds a set of traces, so a node that ran under two toolchains keeps a valid trace for each, and reverting a file re-matches the older trace. A hit is always re-checked against the current world: “stale” means “some observed cell no longer matches”, exactly what the hit test checks.

pp checks authority at the same moment. It serves a stored result only if the caller holds capabilities covering every file cell in the trace's read closure (LAW 23b). Reads propagate to enclosing nodes, so the check is transitive: a narrow caller cannot launder a broad read through a cached aggregator. A capability denial is never cached. Authority gates access to a result, it does not rename it.

4.3 pp why

`pp why file.pp` runs the program and reports, to `stderr`, the fate of every node force: a first-build miss, a verified hit, or a stale trace naming the cell that changed. Here a build reads a file, the same run repeated hits, then you edit the input and the node re-runs:

store-rebuild.sh

```
#!/bin/sh
# A cache hit is decided by re-checking what the node observed, not by a
# timestamp. `pp why` reports each node's fate. A temp HOME keeps the store
# throwaway; ROOT is the canonicalized HOME, stripped from output so the
# transcript is machine-independent (the cell ids are absolute paths).
export HOME=$(mktemp -d)
ROOT=$(cd "$HOME" && pwd -P)
cd "$HOME"

cat > build.pp <<'PP'
let greeting = node { slurp("greeting.txt") }
print(greeting)
PP
echo hello > greeting.txt

echo '$ pp why build.pp --grant fs:$HOME:ro    # first build: a miss'
"$PP" why build.pp --grant "fs:$HOME:ro" 2>&1 | sed "s|$ROOT/||g"

echo
echo '$ pp why build.pp --grant fs:$HOME:ro    # input unchanged: a hit'
"$PP" why build.pp --grant "fs:$HOME:ro" 2>&1 | sed "s|$ROOT/||g"

echo
echo goodbye > greeting.txt
echo '$ pp why build.pp --grant fs:$HOME:ro    # input edited: re-run'
"$PP" why build.pp --grant "fs:$HOME:ro" 2>&1 | sed "s|$ROOT/||g"

rm -rf "$HOME"
```

```
$ pp why build.pp --grant fs:$HOME:ro    # first build: a miss
[why] node 3f73f307115b: miss – no stored trace (first build)
"hello\n"

$ pp why build.pp --grant fs:$HOME:ro    # input unchanged: a hit
[why] node 3f73f307115b: hit – ok trace verified (1 cells)
"hello\n"

$ pp why build.pp --grant fs:$HOME:ro    # input edited: re-run
[why] node 3f73f307115b: trace 1/1 stale – file:greeting.txt changed
[why] node 3f73f307115b: miss – no stored trace usable
"goodbye\n"
```

The node key (f1b010a0e48c here) is stable: it hashes the code, which did not change. The world the trace is verified against changed. First run misses (nothing to verify against); second re-observes the one file cell and hits; the edit makes that cell's hash mismatch, the trace goes stale, no other trace is usable, and the node recomputes.

4.4 Auditing the cache

Two flags let you check the cache instead of trusting it.

--no-cache skips cache reads — every node recomputes, fresh results and traces are still written. A would-be hit reports cache reads disabled (--no-cache).

--check re-runs each missed node's body and compares result hashes. Matching hashes confirm determinism; divergence flags the node volatile and fails the run. This catches a computation that claims to be a pure function of its inputs but is not — the one thing content-addressed caching cannot tolerate.

5 Foreign execution

pp separates computation semantics from execution policy. The language has no compiler, package, container, platform, or Nix form. Libraries build immutable requests as ordinary maps. A trusted host provider decides how to execute one.

run is the ambient process effect: command arguments, inherited host behavior, always scripting-tier — the request does not account for everything that can affect the result.

run-closed! takes immutable tool and input trees, explicit arguments, environment, platform constraints, and selected outputs:

```
run-closed!({
  :tool -> tool-tree,
  :tool-path -> "bin/tool",
  :args -> ["build", "/in/source"],
  :inputs -> source-tree,
  :env -> {},
  :platform -> {"os" -> "linux"},
  :policy -> {:redundancy -> 3},
  :outputs -> ["result"]
})
```

The runtime validates the request and output trees and verifies every blob by hash. It still does not claim that a sandbox makes execution deterministic. Before running a request inside a node, pp asks the installed executor one question: does this exact request account for every semantic input? A trusted provider answers cacheable or scripting-only. pp rejects scripting-only work before the provider executes it.

The bundled Linux provider materializes the declared tool and input trees in a private working directory and supplies only the request's explicit environment to its direct child. This is not OS namespace isolation: absolute filesystem access, the ELF interpreter and shared-library loader, network access, subprocess creation, and other kernel interfaces remain host-mediated. It still exposes clock, randomness, CPU/kernel behavior, and resource limits, so it honestly classifies every request as scripting-only. This makes it useful for portable release actions without making their results silently cacheable.

5.1 Policy belongs to libraries and providers

The classification is the whole core interface; pp does not interpret :platform or :policy. Policy is optional canonical pp data, default {}. A Nix-like provider defines its schema, verifies requests against it, and classifies them cacheable; macros construct the request values. No AST form or evaluator change required.

An application manifest may provide :execution-policy as the default for requests that omit :policy. An explicit request policy always wins. The runtime preserves policy data and asks the trusted executor whether the exact request is cacheable; pp libraries define the policy vocabulary.

stdlib/dune.pp demonstrates the split. dune-build(:working-tree, spec) lets Dune incrementally build an observed development tree and returns a canonical artifact tree. dune-build(:closed-

source, spec) sends the corresponding immutable request to run-closed!. Dune targets, build-directory rules, platform constraints, and output selection stay in the library.

This is the modular boundary: pp supplies lazy nodes, content identity, capabilities, traces, scheduling, and reconciliation. Integrations supply ordinary values and trusted providers.

6 Capabilities and secrets

A pure computation needs no permission. The moment one touches the world — reads a file, runs a process, makes a request — it needs a capability: an unforgeable token saying it is allowed to. No capability, no effect. Not a sandboxing layer bolted on the side; the only way a world-touching effect ever runs.

Here is a program that tries to read a file with no authority at all:

cap-read.pp

```
# Reading a file requires a filesystem capability. This program is granted
# nothing, so the read is refused before it happens — the check is on
# authority, not on whether the file exists. Observation sigils ($file,
# $env, etc.) are the idiomatic way to read the world; a bare `perform
# read-file` is linted.
print($file("/etc/hostname"))

$ pp cap-read.pp
```

```
pp: error: slurp: permission denied for /etc/hostname at cap-read.pp:6
```

The read is refused before it happens; the check is on authority, not on the file's existence or contents — pp never looked. The error names the effect, the missing access, and where the attempt was made.

6.1 Capabilities enter only at the root

No expression creates authority. filesystem in filesystem("/", :rw) is an unbound symbol, not a constructor. The sole mint is the command line: --grant hands the program a capability as it starts. Grant the read above a covering filesystem capability and it succeeds:

cap-grant.sh

```
#!/bin/sh
# The same read-file effect that fails without authority succeeds once the
# root grants a covering capability. Capabilities enter ONLY here, on the
# command line: there is no expression that mints one. A throwaway HOME with
# a file under it keeps the run hermetic; we cd into it and use relative
# paths so nothing machine-specific reaches the output.
export HOME=$(mktemp -d)
mkdir -p "$HOME/etc"
printf 'dbl.internal' > "$HOME/etc/hostname"
cd "$HOME"

cat > read-hostname.pp <<'PP'
print(perform read-file("etc/hostname"))
PP

echo '$ pp read-hostname.pp' # no grant: refused'
"$PP" read-hostname.pp 2>&1

echo
echo '$ pp --grant fs:etc:ro read-hostname.pp' # covering grant: allowed'
"$PP" --grant fs:etc:ro read-hostname.pp 2>&1

rm -rf "$HOME"
```

```
$ pp read-hostname.pp # no grant: refused
pp: error: read-file: capability error: no read access for /tmp/tmp.92CrGPXVXJ/etc/hostname at read-
hostname.pp:1

$ pp --grant fs:etc:ro read-hostname.pp # covering grant: allowed
"dbl.internal"
```

A grant is a scheme, a target, and — for the filesystem — a mode:

- `--grant fs:<path>:ro` (or `:rw`, `:wo`) — read, write, or read-write access to a directory and everything under it. Scope matching is by path component on the canonicalized full path, so `fs:/tmp:ro` covers `/tmp/x` but never `/tmpevil`.
- `--grant net:<host>` or `net:<host>:<port>` — network access to a host, and optionally one port; `net:*` wildcards the host.
- `--grant secret:<path>` — a sealed read of the files under a path (below).
- `--grant process` — the authority to run subprocesses.

Pass `--grant` more than once to hold several capabilities at once.

6.2 User code narrows, never widens

Code holds, passes, and attenuates capabilities — only downward. `current-capabilities()` reifies the ambient authority as of the call: an observation of the ceiling every `perform` checks against, not a mint. Two forms narrow it:

- `cap-restrict(cap, scope)` — restricts `cap` to a sub-scope, optionally to a narrower mode, for example `cap-restrict(cap, "src", :ro)`. Asking for a mode wider than `cap` already grants at that scope is a `Capability_error`, never a silent widen.
- `cap-compose(a, b, ...)` — unions capabilities the code already holds; it can only ever produce authority already present in its arguments.

`with-caps(cap-expr) { body }` replaces ambient authority with `cap-expr` for the extent of `body`, gated by a subset check against the current ambient — so narrowing composes even when another binding lexically holds something broader. Restored on return, tail call, or raise. No union-with-ambient form exists: with capability values in the language, that would be a widening backdoor.

Authority is a ceiling, re-checked at every perform, never part of identity: widening or narrowing the grant renames and invalidates nothing.

6.3 Authority gates the cache, transitively

A cache hit is a message from a past run, so authority gates it too. To serve a stored result `pp` requires the caller's capabilities to cover its transitive read closure: every cell the node read, recursively including child nodes. A caller scoped to `src/` gets no hit on a node whose closure touched `/etc/passwd` — even though the work is done and sitting in the store, being handed the result would reveal the read. Denials are not memoized; granting later still yields the hit.

A node may not carry a capability across its boundary either direction: one as (or inside) a free variable is rejected before keying; one in the result, before storing. Otherwise authority could ride a cached result out to a narrower caller.

6.4 Secrets: sealed cells

A secret needs the opposite guarantee: use a value without its bytes leaking into the terminal or into the store, where any later run could read them back. A read covered by `secret:` — not also by `plain fs:` over the same path — returns a sealed value:

cap-secret.sh

```
#!/bin/sh
# A secret is granted with `secret:` instead of `fs:`. A read covered by a
# secret grant (and NOT by a plain fs grant) yields a SEALED value: it prints
# redacted, its bytes are pinned in memory only and never written to the
# content-addressed store, yet a hash of those bytes still rides in the node's
# trace – so rotating the secret invalidates exactly its observers. Hermetic:
# throwaway HOME, a secret file under it, relative paths.
export HOME=$(mktemp -d)
mkdir -p "$HOME/vault"
printf 'hunter2-swordfish' > "$HOME/vault/token"
cd "$HOME"

# (1) A sealed value prints redacted – the bytes never reach the terminal.
cat > show.pp <<'PP'

print(slurp("vault/token"))

PP
echo '$ pp --grant secret:vault show.pp'
"$PP" --grant secret:vault show.pp 2>&1

# (2) Derive from the secret inside a node (its length), which populates the
# store, then scan the whole store for the plaintext.
echo
cat > derive.pp <<'PP'

print(force(node { string-length(unseal(slurp("vault/token"))) }))

PP
echo '$ pp --grant secret:vault derive.pp      # length of the secret'
"$PP" --grant secret:vault derive.pp 2>&1

echo
echo '$ grep -r hunter2-swordfish ~/.pp/store # the bytes are not there'
if grep -rq 'hunter2-swordfish' "$HOME/.pp/store" 2>/dev/null; then
  echo 'FOUND (leak)'
else
  echo '(no match – secret bytes absent from the store)'
fi

rm -rf "$HOME"
```

```
$ pp --grant secret:vault show.pp
#<sealed>

$ pp --grant secret:vault derive.pp      # length of the secret
17

$ grep -r hunter2-swordfish ~/.pp/store # the bytes are not there
(no match – secret bytes absent from the store)
```

Three things hold at once: the sealed value prints redacted as `#<sealed>`; its bytes pin in memory only and never reach the store (the `grep` above comes up empty); and a hash of the bytes rides in the trace, so rotating the secret invalidates exactly the nodes that observed it while siblings keep hitting.

Deriving from a secret inside a node produces ordinary data — the derived value is no longer the secret. `unseal(v)` is the one explicit way back to a plain string; there is no implicit dataflow tainting,

so confidentiality ends exactly where the program says. If a path is covered by both `secret:` and `fs:` grants, filesystem behaviour wins: granting plain access says “not secret here”.

7 Modules and islands

A module is a block of code whose definitions become a value — packaging is not layered over the language; it is what happens when you take the names a block defined and hand them back as something you can bind, pass, and import. Islands extend that value across machines by pinning it to a content hash.

7.1 Modules

`module { ... }` evaluates its body in a fresh scope and produces a value carrying the names it defined. `import(m)` forces that value and merges those names into the current scope:

mod-module.pp

```
# module { ... } evaluates its body in a fresh scope and produces a value whose  
# fields are the names it defined. import(...) forces that value and merges  
# those names into the current scope. Here the module is bound to `geo`, then  
# imported, so `area` and `pi` become callable.
```

```
let (geo = module {  
  let pi = 3.14159  
  def area(r) { pi * (r * r) }  
}) {  
  import(geo)  
  print(area(10))  
}
```

```
$ pp mod-module.pp
```

```
314.159
```

The body sees only itself, not the surrounding scope: what escapes is exactly what it defined, and only on import. A module value can be held, passed, conditionally imported — ordinary until you pull its names in.

7.2 Loading from files

Two forms bring a file’s definitions in; they differ in whether the file gets its own scope:

- `load("f.pp")` evaluates the file’s forms in the current scope — its definitions merge in directly, as if you had typed them here.
- `load-module("f.pp")` evaluates the file isolated, in its own fresh scope, and returns its exports as a module value — which you then import. The file’s internal scope never leaks into yours.

mod-load-module.sh

```
#!/bin/sh
# Two ways to bring a file's definitions in. `load` MERGES a file into the
# current scope (its defs become directly visible). `load-module` loads the
# same file ISOLATED and returns its exports as a value, which you then
# `import` — so the file's own scope never leaks in. Hermetic throwaway HOME.
export HOME=$(mktemp -d)
cd "$HOME"

cat > mathlib.pp <<'PP'
def square(x) { x * x }
def cube(x) { x * square(x) }
PP

cat > use-load.pp <<'PP'
# load: mathlib's defs merge into this scope.
load("mathlib.pp")
print(square(9))
PP
echo '$ pp use-load.pp          # load merges definitions in'
"$PP" use-load.pp 2>&1

echo
cat > use-load-module.pp <<'PP'
# load-module: mathlib loads isolated; its exports come back as a value.
let (m = load-module("mathlib.pp")) {
  import(m)
  print(cube(3))
}
PP
echo '$ pp use-load-module.pp  # load-module returns exports to import'
"$PP" use-load-module.pp 2>&1

rm -rf "$HOME"
```

```
$ pp use-load.pp          # load merges definitions in
81

$ pp use-load-module.pp  # load-module returns exports to import
27
```

Reach for `load` when the file is a fragment of your program, `load-module` when it is a self-contained unit. Either way the read is the loader's own: runtime authority, bounded to source roots, exempt from user capability accounting. Editing a loaded file still invalidates the nodes that loaded it — the load is recorded in their traces even though it costs no capability.

7.3 Islands

An island is a module that lives elsewhere — another directory, a git repository, a URL — referenced by URI and pinned inline by the content hash of its source tree:

```
island("file:./lib", "e5d7b0f8...")
island("github:owner/repo#main", "a1b2c3...")
```

The pin is part of the code: folding into the expression's hash makes island identity structural — no lockfile, no hidden resolution step; a pinned island form denotes the same bytes wherever pasted. The `#ref` matters only when fetching; the pin argument is always the 64-hex tree hash.

Resolution never touches the network: the pin names an immutable tree under `~/.pp/islands/src/<pin>/` (`entry.pp` is the module root), verified against the pin on every resolve; mismatch is a hard error, as is an unpinned form. Deriving a pin and fetching `git:/github:` trees the first time is the one impure step, opt-in: `pp --update` re-resolves and rewrites pins; `git:/github:` fetching needs `--fetch-islands`. With neither, evaluation stays hermetic.

The workflow, end to end, on a local file: island:

mod-island.sh

```
#!/bin/sh
# An island is a module referenced by URI and pinned INLINE by the content
# hash of its source tree. Resolution never touches the network: the pin names
# an immutable tree in the island cache. An unpinned form is a hard error that
# names the fix; `pp --update` derives the pin and writes it into the source;
# the pinned form then evaluates the tree's entry.pp as a module. Hermetic
# throwaway HOME (which also holds the island cache).
export HOME=$(mktemp -d)
mkdir -p "$HOME/lib"
printf 'def greet(who) { string-append("hello, ", who) }\n' > "$HOME/lib/entry.pp"
cd "$HOME"

cat > app.pp <<'PP'
let (m = island("file:./lib")) {
  import(m)
  print(greet("world"))
}
PP

echo '$ pp island-pins app.pp # the form is unpinned'
"$PP" island-pins app.pp 2>&1

echo
echo '$ pp --update app.pp # derive the pin and write it into the source'
"$PP" --update app.pp 1>/dev/null

echo
echo '$ cat app.pp # the pin is now part of the code'
cat app.pp

echo
echo '$ pp app.pp # resolve the pinned tree and run it'
"$PP" app.pp 2>&1

rm -rf "$HOME"
```

```
$ pp island-pins app.pp # the form is unpinned
file:./lib (unpinned)

$ pp --update app.pp # derive the pin and write it into the source

$ cat app.pp # the pin is now part of the code
let (m = island("file:./lib", "f41331087aee74de14f2e16cbcf21900e8d67f675e1f739383f82fef27070be7")) {
  import(m)
  print(greet("world"))
}

$ pp app.pp # resolve the pinned tree and run it
"hello, world"
```

`pp island-pins` reports each form unpinned, cached, or tampered; `pp --update` fills in a missing pin and refuses to half-rewrite a form it cannot rewrite unambiguously. Once pinned, an island is just another module.

8 Domains and the reconciler

A node computes a value; a build must change the world. You do not mutate it directly — you compute a desired state: a pure, hashable value saying what the world should contain (`{path → content}`, `{service → spec}`). A domain observes the world, diffs against your desired value, applies the minimal change to converge. React’s contract verbatim: return the desired DOM; the reconciler applies the diff.

Convergence follows from the value, not from a maintained script: desired state is a pure function of input cells, so pp caches it, re-derives cheaply on change, and re-observes reality rather than trusting a state file. Drift — a hand-edited file, a dead service — is just a difference the next pass erases.

8.1 A domain is observe / diff / apply

A domain is three ordinary pp functions registered over a namespace of cells:

- `observe : () → value` reads the current world. It runs fresh every pass, never cached.
- `diff : (observed, desired) → plan` computes what to change. It is pure: it runs under an empty capability set and cannot touch the world. Its result is cached, keyed on the diff code plus the observed and desired values.
- `apply : plan → nil` performs the plan’s changes, under the domain’s own write capability.

Core wraps every domain’s apply in one discipline: a journalled intent/done bracket, atomic writes, and verify-after-write. Core re-observes and re-diffs once the apply returns. A plan that still has work left is a hard error. A domain is the single writer for its namespace. So there are no write-write races, and your code needs no ordering discipline. One rule enforces this: stratification. The desired state may not read the domain’s own cells, or reconcile would loop forever. Reading your output tree to decide your output tree is refused.

The trusted mechanics touching the world are a few core primitives: atomic temp-file-plus-rename, fork/exec/reap, a per-domain key-value store (`tree-observe`, `materialize-file`, `remove-file`, `proc-spawn`, ...). Everything else is pp library too: `stdlib/domain-fs.pp` and `stdlib/domain-proc.pp` are real readable source packaging the filesystem and process policies for explicit registration over the core-enforced protocol (create vs update, when to restart). The built-in `--reconcile` and `--supervise` domains implement the same policy natively in the runtime.

8.2 Reconciling a filesystem

`pp --reconcile R00T prog.pp` registers the built-in fs domain with a write capability narrowed to R00T and takes the program’s final canonical tree value as the desired state. File entries carry mode and blob identity; directory entries make parents explicit. It diffs by content hash and materializes missing and changed files atomically; files under R00T that the map does not mention are deleted. An fs write grant over R00T is required; without it nothing is written.

The transcript below reconciles two files into a fresh root, introduces drift by hand (one delete, one edit), and reconciles again: exactly one create and one update, tree restored. The summary names counts per kind, its root path filtered to R00T for machine-independent output.

domain-reconcile.sh

```
#!/bin/sh
# A domain converges the world to a DESIRED state you return as a value. The
# fs domain (stdlib/domain-fs.pp) takes a canonical tree value and
# makes a directory tree match it: creating, updating, and deleting files.
# Hermetic: a throwaway HOME (so the store and journal are fresh) and a root
# under it. The reconcile summary's absolute root path is filtered to ROOT.
export HOME=$(mktemp -d)
ROOT="$HOME/site"

cat > "$HOME/site.pp" <<'PP'
{:tree -> {
  "index.html" -> {:kind -> :file, :mode -> 420, :blob -> blob("<h1>pp</h1>\n")},
  "conf" -> {:kind -> :directory, :mode -> 493},
  "conf/app.txt" -> {:kind -> :file, :mode -> 420, :blob -> blob("mode=prod\n")}
}}
PP

filter() { sed "s#root=[^ ]*#root=ROOT#"; }

echo '$ pp --grant fs:ROOT:rw --reconcile ROOT site.pp # first pass: create'
"$PP" --grant "fs:$ROOT:rw" --reconcile "$ROOT" "$HOME/site.pp" 2>&1 | filter
( cd "$ROOT" && find . -type f | sort )
echo "conf/app.txt: $(cat "$ROOT/conf/app.txt")"

echo
echo '# Introduce drift: delete one file, corrupt another.'
rm "$ROOT/index.html"
echo 'mode=DEV' > "$ROOT/conf/app.txt"

echo '$ pp --grant fs:ROOT:rw --reconcile ROOT site.pp # second pass: restore'
"$PP" --grant "fs:$ROOT:rw" --reconcile "$ROOT" "$HOME/site.pp" 2>&1 | filter
( cd "$ROOT" && find . -type f | sort )
echo "conf/app.txt: $(cat "$ROOT/conf/app.txt")"

rm -rf "$HOME"
```

```
$ pp --grant fs:ROOT:rw --reconcile ROOT site.pp # first pass: create
[reconcile:fs] root=ROOT create=2 update=0 delete=0
./conf/app.txt
./index.html
conf/app.txt: mode=prod

# Introduce drift: delete one file, corrupt another.
$ pp --grant fs:ROOT:rw --reconcile ROOT site.pp # second pass: restore
[reconcile:fs] root=ROOT create=1 update=1 delete=0
./conf/app.txt
./index.html
conf/app.txt: mode=prod
```

Nothing in site.pp says how to converge — no “if missing, create”. It states the desired tree; the diff does the rest. Reverting one file and restoring another are the same mechanism.

File contents are raw blob identities, which diff without loading bytes: `rm -rf` on the tree restores from the store with zero tool re-runs when the desired-state nodes hit.

8.3 Watching, and other domains

`pp --watch --reconcile ROOT prog.pp` runs, reconciles, polls observed cells, and re-runs on change — a controller loop. Every registered domain is re-observed and re-applied each tick, whichever cell changed; cheap when nothing moved because the plan cache turns a no-op pass into a hit. External deletion or drift is caught within one poll interval.

The process domain is the same protocol over a different world: `pp --supervise prog.pp` (usually with `--watch`) takes `{service-name → spec}` and keeps the process table matching: starts missing services, stops removed ones, restarts changed or killed services within a poll interval. Requires `--grant process`; refuses stratification on its own `proc`: cells like the `fs` domain. A third-party domain — any observe/diff/apply triple via `register-domain` — gets the same journal bracket, plan cache, verify-after-write, and stratification for free. The protocol is generic, not filesystem-shaped.

Load `stdlib/domain.pp` for composition helpers such as `domain(spec)`, `probe(name, observe, cap)`, and `register-domains(domains)`. These are ordinary `pp` functions and do not bypass the generic lifecycle.

8.4 Fenced effects

Convergence covers only idempotent change: applying a desired state twice is the same as applying it once. Some actions are not like that, such as sending an email or charging a card. Those may not appear in a node at all. A node is cache-replayable and must never carry an irreversible action. The scripting form `fenced(KIND, SPEC)` registers such an action for the reconciler to run after all convergent work, at most once per pass. It carries an intent/done journal. So a crash mid-action is recovered by policy (`--fenced-policy retry | abort | ask`) rather than by a silent retry that might double-charge. The desired-state law tames the convergent world. Fenced effects are the named carve-out for the part that cannot be converged.

9 Distribution

The thesis of this manual: no local/remote distinction at the language surface. No `remote-eval`, no placement annotation, no node-pinning form — ever. `force` is the only execution primitive; where it runs is a scheduler decision, chosen by flag, never written into the program. A program is byte-identical on one core, eight, or a cluster. The moment location becomes syntax, callers hard-code topology and the deployment boundary is back inside the language.

`--schedule serial | parallel:N | race:N | remote:<member>` selects the policy. `parallel:N` forks workers across cores; `race:N` runs `N` redundant copies and takes the first; `remote:<member>` ships work to another machine. Same feature, different fan-out. The scheduler is result-transparent — it changes where and when, never the answer — so it appears in no cache key and no trace. `pp --check` proves it: any non-serial policy forced serial against the same store must match hashes.

Libraries can select the same handlers through `stdlib/runtime.pp`. They can also provide a pure custom scheduler policy over data-only job descriptors. The policy returns a mode and a complete partition of job indexes; the runtime still owns execution, cancellation, and remote transport.

9.1 Identity is location-independent

Location can be the scheduler's business because identity is a content hash: a node's key hashes its code and input values, never where or when it ran. Two machines evaluating the same node compute the same key and byte-identical result with no coordination. Results move by that hash: published into a shared store, pulled by name, re-hashed on receipt. The content address is the integrity check.

The transcript uses two throwaway HOMEs as two machines with separate stores and a shared directory as transport. Machine A builds a node; machine B, given the same program, computes the identical

key without talking to A. A publishes the result by hash, B pulls it byte-identical; one flipped byte in transit and B rejects it.

dist-by-hash.sh

```
#!/bin/sh
# Distribution in pp is not a language feature: identity is a content hash, so
# a computation has the SAME name on every machine, and a result is moved by
# that hash. Two throwaway HOMEs stand in for two machines (each has its own
# ~/.pp store); a shared directory is the transport (a plain local-dir
# loopback – no network transport ships yet (see docs/THREAT-MODEL-cluster.md).
# Content hashes are deterministic, so they are reproducible output.
export HOME=$(mktemp -d)
A="$HOME/machine-a"; B="$HOME/machine-b"; SHARED="$HOME/shared"
mkdir -p "$A" "$B" "$SHARED"

cat > "$HOME/node.pp" <<'PP'
print(force(node {
  perform log("building greeter.o")
  6 * 7
}))
PP

echo '$ pp node.pp' # on machine A: the node body runs'
HOME="$A" "$PP" "$HOME/node.pp" 2>&1
KEY=$(ls "$A/.pp/store/traces")
RH=$(grep -oE '[0-9a-f]{64,}' "$A/.pp/store/traces/$KEY" | head -1 | tr -d '')
echo "node key: $KEY"
echo "result hash: $RH"

echo
echo '$ pp node.pp' # on machine B, independently: SAME node key'
HOME="$B" "$PP" "$HOME/node.pp" 2>&1
KEY_B=$(ls "$B/.pp/store/traces")
[ "$KEY_B" = "$KEY" ] && echo "machine B computed the same key: yes"

echo
echo '# A publishes the result object by hash; B pulls it by that hash.'
echo '$ pp --transport-push object <hash> shared' # on A'
HOME="$A" "$PP" --transport-push object "$RH" "$SHARED" >/dev/null 2>&1
echo '$ pp --transport-pull object <hash> shared' # on B, re-hash-verified'
HOME="$B" "$PP" --transport-pull object "$RH" "$SHARED" 2>&1
diff -q "$A/.pp/store/objects/$RH" "$B/.pp/store/objects/$RH" >/dev/null \
&& echo "byte-identical across the transport: yes"

echo
echo '# A tampered object is refused on receipt: the hash IS the integrity check.'
printf 'X' | dd of="$SHARED/objects/$RH" bs=1 seek=3 count=1 conv=notrunc 2>/dev/null
echo '$ pp --transport-pull object <hash> shared' # after a byte flips in transit'
HOME="$B" "$PP" --transport-pull object "$RH" "$SHARED" 2>&1

rm -rf "$HOME"
```

```
$ pp node.pp # on machine A: the node body runs
[info] building greeter.o
42
node key: 2e439de4fe0f78cca7d94cefffde961feb12d60279645006f536c2768d1fb6dd
result hash: d98fba09338648079975ad61b2a3ca43cb3ddf4e2c36c7724f66aab333224012
```

```
$ pp node.pp # on machine B, independently: SAME node key
[info] building greeter.o
42
machine B computed the same key: yes
```

```
# A publishes the result object by hash; B pulls it by that hash.
$ pp --transport-push object <hash> shared # on A
$ pp --transport-pull object <hash> shared # on B, re-hash-verified
byte-identical across the transport: yes
```

The machines never agreed on a key; they derived the same one from the code. A build computed “here” is usable “there” because results are addressed by what they are, not where they were produced.

9.2 Transport boundary

The by-hash sync, re-hash-on-receipt, signed-token authority checks, and remote placement are implemented and tested over a local-directory transport. Two separate `~/.pp` stores stand in for two machines. No SSH implementation or generic transport plugin ships today. A remote provider can carry the same canonical artifacts and signed requests; integrity and authority remain runtime checks rather than promises made by the connection.

9.3 Host-qualified identity

Host-qualified identity names world state (“the greeter running on B”), not a dispatch location: a process on host B is a different cell than the same process on host A. The desired-state map generalizes one level, from `{domain → desired}` to `{host → {domain → desired}}`; `--member-name B` selects host B’s slice and hands it to the unchanged reconciler. A member is just `pp --watch --supervise --member-name B`: the single-machine supervisor, verbatim, fed a slice of a cluster-wide value. Naming a host absent from the map is a hard error, not a silent no-op.

Membership is ambient configuration: which machines exist, and where their stores are. It is read from `~/.pp/cluster/members` or `$PP_CLUSTER_MEMBERS`. It is deliberately not a capability. An address is not an authority ceiling, the same distinction the language draws everywhere between location and permission. Authority to act across the cluster travels separately, as a signed token minted from the same `--grant` grammar you use locally.

9.4 Trace merge, audit, and growth

A result carries its verifying trace, so syncing a result syncs its evidence: the pulled `(cell, hash)` observations are re-verified against the local world before a hit. Authority checks survive the crossing — a hit served over the wire is gated on the requesting token’s authority over the trace’s read closure, exactly as locally. A narrow caller cannot launder a broad read through a cached aggregator just because bytes arrived by network, and `pp` why on a pulling machine redacts identically to a purely local run. Audit redacts; it never lies.

A shared store grows, so growth is bounded explicitly. Each pass records its desired object and forced node keys as a wanted root; `pp gc` walks the same durable edges used by validation and stabilization and sweeps only unreachable objects, traces, blobs. Never automatic; a grace period protects concurrent writes, a changed root manifest stops deletion.

10 The evaluator

`pp` runs every program through one explicit continuation evaluator: shared AST interpreted directly, one force path for ephemeral and persistent thunks. Source text is parsed by `pp`’s readers; the host reader is never exposed to a program.

Correctness is checked by expected-output programs, focused property sweeps, and process-level shell scenarios: `stdout`, `stderr`, source ranges, exit status; store persistence, authority, effects, watch loops, transport, reconciliation, crash recovery.

10.1 The expected-output suite

Build a saved image and run the suite from the repository root:

```
scripts/build-lisp.sh --output lisp/pp
scripts/run-tests.sh bin/pp
```

The runner visits every `tests/NNN-*.pp` program and diffs its output against the committed `tests/NNN-*.pp.expected` file. A missing `.expected` file is a failure. Shell suites (`tests/*.sh`) exercise multi-process scenarios with isolated stores and homes.

10.2 Kernel properties

`pp --check-kernel-props` runs deterministic sweeps for hash injectivity, quotation, print round-trips, and capability algebra. The generated structures are checked exhaustively, so a new constructor requires a corresponding property case.

```
bin/pp --check-kernel-props --seed 1 --count 3000
```

11 Language reference

Full detail on the language chapter's summary. Each section covers one construct and shows it running. `pp` is a Lisp-1: one namespace for functions and variables; inner-first, strict evaluation.

The special forms — the syntax the reader treats specially, rather than ordinary function calls — are: `if`, `do`, `let`, `let*`, `fn`, `def`, `node`, `match`, `quote`, `quasiquote` with `unquote` and `splice`, `and`, `or`, `delay`, `force`, `perform`, `with-handler`, `with-caps`, `with-config`, `config`, `assert`, `module`, `import`, `load`, `load-module`, `island`, `reconcile`, `try`, and the `:` type annotation. `defmacro` is recognized structurally at the top level; `needs` modifies a named node definition. Everything else is a function. In the s-expression surface, named node forms are represented as `defnode`, and `splice` is represented as `unquote-splicing`.

The brace surface also supplies infix operators, pipelines (`|>`), f-strings, `spread (...)`, observation sigils (`$file`, `$env`, `$glob`, `$probe`, `$secret`, `$config`), and the `with { caps: ..., config: ..., handlers: ... }` clause form. Their exact lowerings are listed in the syntax appendix of `SPEC.md`.

11.1 Value types

Integers, floats, strings, the booleans `true` and `false`, `nil`, keywords (`:name`), and quoted symbols (`quote { name }`) are the atoms. `print` shows strings quoted and prints one value per call.

ref-atoms.pp

```
# Integers, floats, strings, booleans, nil, keywords, and quoted symbols.
# `print` shows strings quoted; everything else prints as it reads.
print(42)
print(3.14)
print("text")
print(true)
print(false)
print(nil)
print(:keyword)
print(quote { sym })
```

```
$ pp ref-atoms.pp
```

```
42
3.14
"text"
true
false
nil
:keyword
sym
```

The collections are lists, vectors, maps, and sets, each with its own printed form.

ref-collections.pp

```
# Lists, vectors, maps, and sets – each with its own literal syntax.
print([1, 2, 3])
print(vec[1, 2, 3])
print({:a -> 1, :b -> 2})
print(hash-set(1, 2, 3))

$ pp ref-collections.pp
```

```
(1 2 3)
[1 2 3]
{:a 1, :b 2}
#{1 3 2}
```

11.2 Arithmetic and comparison

+ and * are variadic. - and / take exactly two arguments. mod is the integer remainder.

ref-arith.pp

```
# '+' and '*' are variadic; '-' and '/' take exactly two arguments; 'mod'
# is the remainder. Comparisons chain left to right: '<(1, 2, 3)' is
# '1 < 2 AND 2 < 3'.
print(+(1, 2, 3))
print(10 - 3)
print(*(2, 3, 4))
print(20 / 4)
print(17 mod 5)

$ pp ref-arith.pp
```

```
6
7
24
5
2
```

Comparisons are variadic and chain left to right — `<(1, 2, 3)` means `1 < 2` and `2 < 3`.

ref-compare.pp

```
# Comparisons are variadic and chain left to right.
print(2 = 2)
print(<(1, 2, 3))
print(>(3, 2, 1))
print(<=(1, 1, 2))
print(3 >= 3)

$ pp ref-compare.pp
```

```
true
true
true
true
true
```

11.3 Booleans and conditionals

Only `nil` and `false` are falsy; every other value, including `0`, is truthy. `and` and `or` short-circuit and return the deciding value rather than a boolean.

ref-bool.pp

```
# Only nil and false are falsy; everything else (including 0) is truthy.
# `and`/`or` short-circuit and return the deciding value, not a boolean.
# Flat `else if` chains keep multi-way branching clean.
print(if 1 { if 2 { 3 } else { false } } else { false })
print(if false { true } else if nil { true } else { 5 })
print(if true { false } else { false })
print(not(nil))
print(not(0))
```

```
$ pp ref-bool.pp
```

```
3
5
false
true
false
```

`if` is an expression that evaluates exactly one branch. The untaken branch never runs — no effects, no errors.

ref-if.pp

```
# `if` is an expression and evaluates exactly one branch — the untaken one
# never runs, so the error here is never raised.
print(if 5 > 0 { "pos" } else { "neg" })
print(if false { error("never") } else { "safe" })
```

```
$ pp ref-if.pp
```

```
"pos"
"safe"
```

11.4 Bindings

`let` bindings are mutual: every binding is visible in every right-hand side and in the body, regardless of the order written.

ref-let.pp

```
# `let` bindings are mutual: every binding sees every other, whatever the
# textual order. One flat `let` with all bindings is the idiomatic style.
print(let (y = x + 1, x = 1) { y })
```

```
$ pp ref-let.pp
```

```
2
```

`let*` is the sequential form. Each binding sees only the ones above it, so a name may shadow itself.

ref-letstar.pp

```
# `let*` is sequential: each binding sees only the ones written above it,  
# so a name can shadow itself.  
print(let* (x = 1, x = x + 1) { x })  
  
$ pp ref-letstar.pp
```

```
2
```

At the top level, names are predeclared across the scope. A function definition can refer to a later definition; a value definition still evaluates its value at its statement, so a later value is not available until that statement runs.

ref-def.pp

```
# At the top level, names are predeclared across the scope. Function definitions  
# can refer forward; value definitions still evaluate at their statement.  
let answer = 6 * 7  
print(answer)  
  
def square(x) { x * x }  
print(square(7))  
  
$ pp ref-def.pp
```

```
42  
49
```

11.5 Functions

`fn` produces an anonymous function; `def` with a list head names one. There is no `loop` keyword — recursion is the loop, and tail calls run in constant stack.

ref-fn.pp

```
# `fn` makes an anonymous function; recursion is the only loop.  
print((fn(x) { x + 1 })(41))  
  
def factorial(n) {  
  if n = 0 { 1 } else { n * factorial(n - 1) }  
}  
  
print(factorial(5))  
  
$ pp ref-fn.pp
```

```
42  
120
```

11.6 Nodes

`node { ... }` creates a persistent, content-addressed computation. Its body runs when forced, and its result can be reused across processes. A named, zero-argument node uses `node build { ... }` and binds a node thunk. A parameterized node `compile(src) { ... }` is the brace spelling of the s-expression `defnode` form; applied named-node keying remains a documented LAW 6 follow-up, so

the current persistent keying guarantee is for node thunks, not an argument-keyed application cache. needs narrows the capabilities available to the node body, for example `node read(path) needs fs.read(path) { ... }`.

The node key contains the body and the values of the free variables it reads; ambient capabilities, handlers, and config are not identity inputs.

node-identity.pp

```
# A node is identified by its code and its inputs (LAW 20). Two nodes with
# identical code and free-variable values are the SAME node: the first force
# runs the body and stores the result; the second finds it already in the
# store. A store hit does not replay the body's output (LAW 17), so
# "compiling" prints exactly once.
let first = node { print("compiling"); 6 * 7 }
let second = node { print("compiling"); 6 * 7 }
print(first)
print(second)

$ pp node-identity.pp
```

```
"compiling"
42
42
```

11.7 Sequencing

do forces each form in order for its effects and returns the value of the last.

ref-do.pp

```
# `do` runs each form in order for its effects and returns the last value.
print(do {
  print("step 1")
  print("step 2")
  99
})

$ pp ref-do.pp
```

```
"step 1"
"step 2"
99
```

11.8 Type annotations

Annotations are optional gradual claims, written with `:`, and checked when the function's body runs — not before. A well-typed call passes through unchanged.

ref-types.pp

```
# Type annotations are optional and checked when the body runs. A well-typed
# call passes through unchanged.
def increment(n: int): int { n + 1 }
print(increment(41))

$ pp ref-types.pp
```

42

A mismatch names the offending value and its source location.

ref-type-error.pp

```
# A type annotation mismatch names the offending value and location.
def increment(n: int): int { n + 1 }
increment("oops")

$ pp ref-type-error.pp
```

```
pp: error: type mismatch: expected int, got "oops" at ref-type-error.pp:2
```

11.9 Quotation

Braces are the surface syntax; s-expressions are the AST. `quote { ... }` is the bridge: it turns its one form into that AST as data. Homoiconicity lives at the AST layer (Elixir's position): the brace text is not a data structure, but the tree it reads to is, and quotation is total — any form a reader accepts becomes data.

Quasiquote builds structure with holes. The body of `quasiquote { ... }` is a template written in ordinary brace syntax, denoting the s-expression data it reads to; `unquote(e)` fills one hole with a computed value, `splice(e)` splices a list into a list position.

ref-quote.pp

```
# Braces are the surface; s-expressions are the AST. `quote { ... }` turns
# the form inside into that AST, as data. `quasiquote { ... }` is a template:
# `unquote(e)` fills one hole, `splice(e)` splices a list into a list position.
print(quote { if a { b } else { c } })
print(quasiquote { f(1, unquote(1 + 1), splice(list(3, 4))) })

$ pp ref-quote.pp
```

```
(if a b c)
(f 1 2 3 4)
```

11.10 Laziness

`delay` suspends a computation; `force` runs it and memoizes the result, so the body runs at most once however many times it is forced. `force` is the identity on non-thunks.

ref-delay.pp

```
# `delay` suspends a computation; `force` runs it and memoizes the result,  
# so the body runs at most once however many times it is forced.  
let (t = delay(do {  
  print("run once")  
  42  
})) {  
  print(force(t))  
  print(force(t))  
}  
  
$ pp ref-delay.pp
```

```
"run once"  
42  
42
```

11.11 Effects and config

perform dispatches an effect to the nearest handler installed by with-handler. The clause form with { handlers: { :name -> fn(...) { ... } } } { ... } installs the same handler map. Handling an effect yourself needs no capability. Built-in world effects include read-file, write-file, run, run-closed!, http-get, http-post, log, tree-observe, the process effects, and the domain-state effects; user handlers may define additional effect names. Ambient process and network effects are scripting-tier. run-closed! is cacheable only when its installed provider classifies the exact request as such.

ref-perform.pp

```
# `perform` dispatches to the nearest handler installed by `with-handler`.  
# The `handlers:` clause takes a map of keyword -> function pairs.  
with { handlers: { :double -> fn(x) { x * 2 } } } {  
  print(perform double(21))  
}  
  
$ pp ref-perform.pp
```

```
42
```

Config is ambient, dynamically-scoped data, distinct from capabilities. with-config(map) { ... } installs a temporary config frame, and config reads a key with an optional default for when it is absent. with-caps(cap) { ... } narrows the current capability set for the body's dynamic extent; it cannot widen authority.

ref-config.pp

```
# Config is ambient, dynamically-scoped data. ` $config(key)` reads a key,  
# with an optional default when the key is absent. It records a `config:`  
# trace cell so a node that observed config recomputes when it changes.  
print(with-config({:host -> "db1", :port -> 5432}) { $config(:host) })  
print(with-config({:host -> "db1"}) { $config(:timeout, 30) })  
  
$ pp ref-config.pp
```

```
"db1"  
30
```

11.12 Assertions

`assert(condition)` raises a located error when the condition is false. `assert(condition, message)` uses the supplied message. It is syntax sugar for an `if` that returns `nil` on success and calls `error` on failure.

ref-assert.pp

```
# `assert` succeeds silently when its condition is true and returns nil.  
assert(1 = 1)  
print("ok")  
  
$ pp ref-assert.pp
```

```
"ok"
```

11.13 Macros

`defmacro` defines a macro: it receives its arguments as unevaluated forms — s-expression data, the same trees `quote` yields — and returns a new form, expanded before the evaluator sees it. You write the macro in braces; it consumes and produces the AST. A `quasiquote { ... }` template is the usual way to assemble the expansion: ordinary brace syntax with `unquote(e)` holes where the caller's forms go.

ref-defmacro.pp

```
# `defmacro` receives its arguments as unevaluated forms (sexpr data) and  
# returns a new form, expanded before the evaluator sees it. The body of  
# `quasiquote { ... }` is a template in ordinary brace syntax.  
defmacro unless(test, body) {  
  quasiquote { if unquote(test) { nil } else { unquote(body) } }  
}  
  
print(unless(false, "ran"))  
print(unless(true, "ran"))  
  
$ pp ref-defmacro.pp
```

```
"ran"  
nil
```

Because a macro's real domain is the AST, the s-expression notation remains a first-class way to write one: a `.pp1` file is the same language in AST-native form — `pp` reads it with the s-expression reader, and the template `(if ,test nil ,body)` there builds the identical tree the brace template above does.

Data operations work on either origin: where quasiquote gets awkward, assemble the form directly with `list/cons` — `list(quote { + }, x, x)` is a complete macro body. `gensym` supplies fresh names for any binding a macro introduces; `pp` macros are unhygienic, so the discipline is manual.

11.14 Modules

A module is a block whose definitions become a value. `import` merges that value's exports into the current scope; a module's own body is a fresh scope.

ref-module.pp

```
# A module is a block whose definitions become a value; `import` merges that
# value's exports into the current scope. The module's body is a fresh scope.
let (geometry = module {
  def double(x) { x * 2 }
  let tau = 6
}) {
  import(geometry)
  print(double(tau))
}

$ pp ref-module.pp
```

12

`load("path.pp")` evaluates a file in the current scope. `load-module("path.pp")` evaluates it in a fresh scope and returns its definitions as a module value. `island(uri, pin)` imports a content-addressed module tree; an unpinning island is an error. The CLI and workflow details are in the Modules and islands chapter.

11.15 Desired state

`reconcile { key -> value, ... }` is identity-preserving sugar for the final desired-state map consumed by `--reconcile`; it does not perform reconciliation inside the language.

ref-reconcile.pp

```
# `reconcile` names the desired final map; the CLI reconciler consumes it.
print(reconcile { "hello.txt" -> "hello" })

$ pp ref-reconcile.pp
```

{"hello.txt" "hello"}

11.16 Lists

The core list operations are builtins and need no library: `list`, `cons`, `car`, `cdr`, and `map`.

ref-list-ops.pp

```
# List builtins need no library: list, cons, car, cdr, and map are primitive.
# `map` is deliberately lazy in the elements, serving as the parallel
# fan-out point the scheduler batches on.
print(list(1, 2, 3))
print(cons(0, list(1, 2)))
print(car(list(1, 2, 3)))
print(cdr(list(1, 2, 3)))

# map takes the function first – map(f, list) – and is lazy in the elements,
# so it is the point the scheduler batches its parallel fan-out on.
print(map(fn(x) { x * x }, list(1, 2, 3, 4)))

$ pp ref-list-ops.pp
```

```
(1 2 3)
(0 1 2)
1
(2 3)
(1 4 9 16)
```

stdlib/list.pp adds the higher-order functions — foldl, foldr, filter, range, take, drop, reverse, length, nth, append, member?, and each. A program loads it with load("stdlib/list.pp").

ref-list-stdlib.sh

```
#!/bin/sh
# stdlib/list.pp adds the higher-order list functions. It resolves relative to
# the repo root, so the script runs pp from there.
export HOME=$(mktemp -d)
cd "$(dirname "$(dirname "$PP")")"

"$PP" /dev/stdin <<'PP'

(load "stdlib/list.pp")
(print (range 1 6))
(print (foldl + 0 (range 1 6)))
(print (filter (fn (x) (> x 2)) (list 1 2 3 4)))
(print (take 2 (list 10 20 30 40)))
(print (drop 2 (list 10 20 30 40)))
(print (reverse (list 1 2 3)))
(print (length (list 1 2 3 4)))
(print (nth 1 (list 10 20 30)))
(print (append (list 1 2) (list 3 4)))
(print (member? 2 (list 1 2 3)))

PP

rm -rf "$HOME"
```

```
pp: error: unsupported command or source input: /dev/stdin
pp v0.2.0-dev
Usage: pp --version | pp --help | pp -e EXPR | pp [--once] FILE.pp|FILE.ppl | pp build pp [--repro-check]

Language commands:
  pp -e '<brace expression>'
  pp [--once] <file.pp|file.ppl>
  pp run <file.pp|file.ppl>
  pp (stdin/REPL reads brace forms)

Frontend commands:
  pp fmt --to-braces <file.ppl> [-i]
  pp fmt --to-sexpr <file.pp> [-i]
  pp --emit-braces <file.ppl>
  pp --roundtrip-braces <file.ppl>
  pp --compare-hash <file1.pp> <file2.pp>
  pp --list-comments sexpr|brace <file>
  pp lint <file.pp>
  pp --dump-surface-tables

Admin and verification commands:
  pp --dump-builtins
  pp --check-kernel-props [--seed N] [--count N]
  pp cluster-init
  pp --mint-token OUT TTL-SECS [--grant SPEC]
  pp --transport-push|--transport-pull KIND HASH ROOT
  pp --serve-hit KEY TOKEN-FILE SHARED-ROOT REPLY-FILE
  pp --recv-hit REPLY-FILE SHARED-ROOT
```

11.17 List and vector literals

The brace surface distinguishes lists from vectors at the literal level. A bare `[...]` produces a list, while `vec[...]` produces a vector. Both support spread to splice an existing collection:

Form	Meaning
<code>[a, b, c]</code>	List literal (list a b c)
<code>vec[a, b, c]</code>	Vector literal (vector a b c)

Form	Meaning
[head, ...tail]	Spread: cons(head, tail) — only at trailing position

ref-list-vec-literals.pp

List and vector literals — brackets make a list.

```
print([1, 2, 3])
print(vec[1, 2, 3])
```

```
$ pp ref-list-vec-literals.pp
```

```
(1 2 3)
[1 2 3]
```

11.18 Maps

Maps are immutable. The builtins construct one (hash-map), look up a key (hash-map-get), and return new maps or their components (map-insert, map-remove, map-keys, map-vals).

ref-map-ops.pp

Map builtins: construct, look up, insert, and read keys/values. Maps are

immutable — `map-insert` and `map-remove` return new maps.

The bracket syntax `m[:key]` lowers to `hash-map-get`.

```
let scores = {:alice -> 95, :bob -> 87}
print(scores[:alice])
```

Map update via spread — `{ ...scores, :charlie -> 78 }` produces a new map.

```
print({ ...scores, :charlie -> 78 })
print(map-keys(scores))
print(map-vals(scores))
print(map-remove(scores, :alice))
```

```
$ pp ref-map-ops.pp
```

```
95
{:charlie 78, :bob 87, :alice 95}
(:bob :alice)
(87 95)
{:bob 87}
```

stdlib/map.pp adds map-has? and map-merge on top of those builtins; it depends on stdlib/list.pp, so load that first.

ref-map-stdlib.sh

```
#!/bin/sh
# stdlib/map.pp builds on the map builtins and on stdlib/list.pp, so load that
# first.
export HOME=$(mktemp -d)
cd "$(dirname "$(dirname "$PP")")"

"$PP" /dev/stdin <<'PP'

(load "stdlib/list.pp")
(load "stdlib/map.pp")
(def m (hash-map :a 1 :b 2))
(print (map-has? m :a))
(print (map-has? m :z))
(print (map-merge m (hash-map :b 9 :c 3)))

PP

rm -rf "$HOME"
```

```
pp: error: unsupported command or source input: /dev/stdin
pp v0.2.0-dev
Usage: pp --version | pp --help | pp -e EXPR | pp [--once] FILE.pp|FILE.ppl | pp build pp [--repro-check]

Language commands:
  pp -e '<brace expression>'
  pp [--once] <file.pp|file.ppl>
  pp run <file.pp|file.ppl>
  pp (stdin/REPL reads brace forms)

Frontend commands:
  pp fmt --to-braces <file.ppl> [-i]
  pp fmt --to-sexpr <file.pp> [-i]
  pp --emit-braces <file.ppl>
  pp --roundtrip-braces <file.ppl>
  pp --compare-hash <file1.pp> <file2.pp>
  pp --list-comments sexpr|brace <file>
  pp lint <file.pp>
  pp --dump-surface-tables

Admin and verification commands:
  pp --dump-builtins
  pp --check-kernel-props [--seed N] [--count N]
  pp cluster-init
  pp --mint-token OUT TTL-SECS [--grant SPEC]
  pp --transport-push|--transport-pull KIND HASH ROOT
  pp --serve-hit KEY TOKEN-FILE SHARED-ROOT REPLY-FILE
  pp --recv-hit REPLY-FILE SHARED-ROOT
```

11.19 Index access

Bracket indexing works on maps and vectors, lowering to the appropriate builtin. A numeric key calls `vector-get`; any other key calls `hash-map-get`.

Form	Meaning
<code>m[:key]</code>	<code>hash-map-get(m, :key)</code> — map lookup
<code>v[0]</code>	<code>vector-get(v, 0)</code> — vector index

ref-index-access.pp

```
# Bracket indexing on maps and vectors.
let scores = {:alice -> 95, :bob -> 87}
print(scores[:alice])

let vec = vec[10, 20, 30]
print(vec[1])

$ pp ref-index-access.pp
```

```
95
[1]
```

11.20 Map update

The update syntax produces a new map with one or more keys inserted or replaced, without mutating the original.

Form	Meaning
<code>{ ...m, :k1 -> v1, :k2 -> v2 }</code>	Spread update: <code>map-insert(map-merge(m, new-map), k2, v2)</code> — rightmost wins

ref-map-update.pp

```
# Map update via spread — { ...base, key -> value } produces a new map
# with the given keys inserted or replaced.
let base = {:name -> "app", :port -> 8080}
let updated = { ...base, :port -> 9090, :debug -> true }
print(updated[:port])
print(updated[:debug])

$ pp ref-map-update.pp
```

```
9090
true
```

11.21 Strings

The string builtins cover appending, length, splitting, substrings, index-of, trimming, and conversion to and from numbers.

ref-string-ops.pp

```
# String builtins: append, length, split, substring, index-of, trim, and
# number conversion. f-strings (`f"...`) provide interpolation.
print(string-append("foo", "bar"))
print(string-length("hello"))
print(string-split("a,b,c", ","))
print(string-sub("hello", 1, 3))
print(string-index("hello", "ll"))
print(string-trim("  hi  "))
print(number->string(42))
print(string->number("3.14"))
```

```
$ pp ref-string-ops.pp
```

```
"foobar"
5
("a" "b" "c")
"ell"
2
"hi"
"42"
3.14
```

stdlib/string.pp adds string-join, starts-with?, ends-with?, and lines.

ref-string-stdlib.sh

```
#!/bin/sh
# stdlib/string.pp adds string helpers over the string builtins.
export HOME=$(mktemp -d)
cd "$(dirname "$(dirname "$PP")")"

"$PP" /dev/stdin <<'PP'

(load "stdlib/string.pp")
(print (string-join ", " (list "a" "b" "c")))
(print (starts-with? "hello" "he"))
(print (ends-with? "hello" "lo"))
(print (lines "a
b
c"))
PP

rm -rf "$HOME"
```

```
pp: error: unsupported command or source input: /dev/stdin
pp v0.2.0-dev
Usage: pp --version | pp --help | pp -e EXPR | pp [--once] FILE.pp|FILE.ppl | pp build pp [--repro-check]

Language commands:
  pp -e '<brace expression>'
  pp [--once] <file.pp|file.ppl>
  pp run <file.pp|file.ppl>
  pp (stdin/REPL reads brace forms)

Frontend commands:
  pp fmt --to-braces <file.ppl> [-i]
  pp fmt --to-sexpr <file.pp> [-i]
  pp --emit-braces <file.ppl>
  pp --roundtrip-braces <file.ppl>
  pp --compare-hash <file1.pp> <file2.pp>
  pp --list-comments sexpr|brace <file>
  pp lint <file.pp>
  pp --dump-surface-tables

Admin and verification commands:
  pp --dump-builtins
  pp --check-kernel-props [--seed N] [--count N]
  pp cluster-init
  pp --mint-token OUT TTL-SECS [--grant SPEC]
  pp --transport-push|--transport-pull KIND HASH ROOT
  pp --serve-hit KEY TOKEN-FILE SHARED-ROOT REPLY-FILE
  pp --recv-hit REPLY-FILE SHARED-ROOT
```

11.21.1 Interpolation (f-strings)

Interpolation requires the `f` prefix glued to the opening quote. `{expr}` holes take arbitrary expressions and lower through the generic `->string` conversion; the rest of the string is literal text. Ordinary `"..."` strings never interpolate, so `"{x}"` is three literal characters — JSON fragments, shell snippets, and generated code are safe by default.

Form	Meaning
<code>f"a{e}b"</code>	<code>string-append("a", ->string(e), "b")</code>
<code>f"{e}"</code>	<code>->string(e)</code> — coerces any value to its string form
<code>f"{{" / f"}}"</code>	A literal <code>{ / }</code> — doubling escapes the brace
<code>"{x}"</code>	Three literal characters — an ordinary string never interpolates

```
let name = "world"
let n = 3
print(f"Hello, {name}! Built {n} targets.")
```

->string renders a string as itself (no quotes) and every other value in its display form. f-strings desugar to string-append/->string with no dedicated AST node, so they round-trip through pp fmt with the hash preserved.

ref-fstrings.pp

```
# f-strings interpolate with the `f` prefix: `f"Hello, {name}!"`.
# Ordinary `"...` strings never interpolate – `{x}` is three literal chars.
# `->string` coerces any value; a string renders as itself (no quotes).
let name = "world"
let count = 3
print(f"Hello, {name}! Built {count} targets.")
print(f"formula: {2 + 2}")
print(f"just-text")

$ pp ref-fstrings.pp
```

```
"Hello, world! Built 3 targets."
"formula: 4"
"just-text"
```

11.22 Type predicates

Each predicate forces its argument and tests its shape. The full set is int?, float?, string?, bool?, keyword?, symbol?, pair?, vector?, map?, set?, and fn?.

ref-predicates.pp

```
# Type predicates force their argument and test its shape.
print(int?(3))
print(string?("x"))
print(keyword?(:k))
print(pair?(list(1)))
print(map?({:a -> 1}))
print(fn?(car))
print(vector?(vec[1]))

$ pp ref-predicates.pp
```

```
true
true
true
true
true
true
true
```

11.23 Pattern matching

match is the one pattern-dispatch form (there is no cond and there are no function clauses). It tries each arm's pattern against the scrutinee in order; the first that matches wins, and a scrutinee that matches no arm is a runtime error. Patterns are literals, variables (which bind), the wildcard `_`, list patterns with spread (`[a, ...rest]`), and tagged patterns (`[:ok, v]`).

Form	Meaning
<code>match e { p => r }</code>	Bind <code>p</code> 's variables and evaluate <code>r</code> on the first matching arm
<code>p if cond => r</code>	Guard: the arm fires only when <code>p</code> matches AND <code>cond</code> (evaluated under <code>p</code> 's bindings) is truthy, else control falls to the next arm
<code>[a, b, ...rest] => r</code>	List pattern with spread — <code>a</code> , <code>b</code> , and the remainder <code>rest</code>
<code>[:ok, v] => r</code>	Tagged pattern — a two-element list headed by a keyword
<code>_ => r</code>	Wildcard — matches anything without binding

```
def classify(n) {
  match n {
    x if x < 0 => "negative"
    0          => "zero"
    x if x > 100 => "large"
    _          => "small"
  }
}
```

Guards subsume the multi-way conditional: a match on the scrutinized value with guarded arms replaces what a `cond` chain would have spelled. The engine agrees on every pattern kind, and the lowering uses unshadowable internal primitives — redefining `car` or `=` cannot change match semantics. The s-expression surface reads and writes the same form (`(match e (p body) (p if guard body) ...)`), so match files round-trip through `pp fmt`.

ref-match-guards.pp

```
# `match` is the one pattern-dispatch form. Guards (`p if cond => body`)
# let an arm fire only when the pattern matches AND the guard is truthy.
def classify(n) {
  match n {
    x if x < 0  => "negative"
    0          => "zero"
    x if x > 100 => "large"
    _          => "small"
  }
}

print(classify(-5))
print(classify(0))
print(classify(50))
print(classify(200))
```

```
$ pp ref-match-guards.pp
```

```
"negative"
"zero"
"small"
"large"
```

11.24 Spread

The `...` prefix is one concept in three places: list/vector construction, map update and merge (the Maps section above), and call arguments. In a list or vector literal it splices a collection in; in a call it splices arguments into the call, so a spread may appear anywhere in the argument list.

Form	Meaning
<code>[a, ...rest]</code>	<code>cons(a, rest)</code>
<code>vec[a, ...rest]</code>	<code>cons(a, rest)</code> in vector context
<code>f(a, ...rest, b)</code>	Call spread: <code>apply(f, list(a), rest, list(b))</code> — a spread may appear anywhere in the arguments

```
let flags = ["-02", "-Wall"]
run!("cc", ...flags, "-c", src, "-o", obj)
```

A spread whose target is a compound expression uses the spaced form (`... expr`), matching the list-literal spelling.

ref-call-spread.pp

```
# The `...` prefix splices a collection into a function call:
# `f(a, ...rest, b)` lowers to `apply(f, list(a), rest, list(b))`.
# A compound spread target uses the spaced `... expr` form.
# Plain arguments before and after the spread are grouped into `list(...)`
# segments; each spread is its own segment. Here `...parts` splices the list
# into the call so `print` receives five values instead of three.
let parts = ["c", "d"]
print("a", "b", ...parts, "e")

$ pp ref-call-spread.pp
```

```
"a""b""c""d""e"
```

11.25 Observation sigils

The `$` sigil introduces world observations — reads that cross the boundary between pure computation and the outside world. Every observation records the corresponding trace cell, making it visible to the cache-validity system.

Form	Lowering	Trace cell
<code>\$file("path")</code>	<code>slurp("path")</code>	file:
<code>\$env("VAR")</code>	<code>env-get("VAR")</code>	env:
<code>\$env("VAR", "default")</code>	<code>if nil?(env-get("VAR")) "default" env-get("VAR")</code>	env:
<code>\$glob("pattern")</code>	<code>perform tree-observe("pattern")</code>	tree:
<code>\$probe("name")</code>	<code>probe("name")</code>	probe:
<code>\$secret("path")</code>	<code>slurp("path")</code>	sealed:

ref-sigils.pp

```
# Observation sigils read the world and record trace cells.
# Each is a $ prefixed form that lowers to the corresponding builtin.
#
# $file("path")      -> slurp, records file: cell
# $env("VAR")        -> env-get, records env: cell
# $glob("pattern")    -> perform tree-observe, records tree: cell
# $probe("name")      -> probe, records probe: cell
# $config(:key)       -> config, records config: cell
# $secret("path")     -> slurp under secret grant, records sealed: cell
#
# (These require the appropriate --grant at the command line;
#  this example only demonstrates the syntax, not actual execution.)
print("sigils: $file, $env, $config, $probe, $secret")

$ pp ref-sigils.pp
```

```
"sigils: $file, $env, $config, $probe, $secret"
```

11.26 Error propagation: try

The `try` block introduces a region where bindings `unwrap [:ok, v] / [:err, e]` pairs automatically. Each `name <- expr` extracts the value on success or propagates the error on failure. A plain `let name = expr` inside `try` is an ordinary sequential binding that does not unwrap.

The block's last expression is the overall value (reachable only when every binding succeeded).

Form	Meaning
<code>try { a <- f(); body }</code>	Bind <code>a</code> to the unwrapped value; propagate on <code>:err</code>
<code>try { body }</code>	Plain body — no unwrapping, evaluates as usual

ref-try.pp

```
# try blocks unwrap [:ok, v] / [:err, e] pairs automatically.
# Each `name <- expr` extracts the value on success or propagates
# the error on failure. The block's last expression is the overall value.
let ok-result = [:ok, 42]
let err-result = [:err, "oops"]

let (good = try { a <- ok-result; a + 1 }) { print(good) }
let (bad = try { a <- err-result; a + 1 }) { print(bad) }

$ pp ref-try.pp
```

```
43
(:err "oops")
```

11.27 Collecting results

`collect` is a plain function used in pipelines: it partitions a list of `[:ok, v] / [:err, e]` results, returning `[:ok, values]` if every element succeeded or `[:err, errors]` if any failed. It is the validation counterpart to `try` — where `try` short-circuits at the first error, `collect` runs everything and accumulates. There is no `collect { }` block form.

Form	Meaning
<code>srcs > map(f) > collect</code>	<code>[:ok, [v...]]</code> if all ok, else <code>[:err, [e...]]</code>

ref-collect.pp

```
# collect partitions a list of [:ok, v] / [:err, e] results.
# It returns [:ok, values] if all succeeded or [:err, errors] if any failed.
let results = list([:ok, 1], [:ok, 2], [:err, "x"])
print(results |> collect)

let all-ok = list([:ok, 10], [:ok, 20])
print(all-ok |> collect)

$ pp ref-collect.pp
```

```
(:err ("x"))
(:ok (10 20))
```

12 Command-line reference

The flags `pp` accepts, grouped by use; from `bin/pp --help`, which wins on disagreement. Flags marked internal are dispatch machinery `pp` invokes on itself — listed for completeness.

Flags compose: `--grant`, `--schedule`, and `--watch` layer onto any run mode. Anything after a bare `--` becomes the program's `argv`, read with `argv()`.

12.1 Running programs

Invocation	Meaning
<code>pp</code>	Start the REPL (tree-walker).
<code>pp <file.pp></code>	Read and run a source file; the program's value is its last top-level form.
<code>pp run <file></code>	Same as <code>pp <file></code> — <code>run</code> is an explicit subcommand spelling.
<code>pp -e '<expr>'</code>	Evaluate one expression string and print each top-level value.
<code>pp --once <file.pp></code>	Run once and exit. A no-op: this is already the default; the flag is for symmetry with <code>--watch</code> .
<code>pp -- <args...></code>	Everything after <code>--</code> becomes the program's <code>argv</code> , read via the <code>argv()</code> builtin.
<code>pp --version</code> , <code>pp -v</code>	Print the version and exit.
<code>pp --help</code> , <code>pp -h</code>	Print the usage summary and exit.

12.2 Execution modes

`pp` has a single tree-walking interpreter over the content-addressed store. Tail calls run in constant stack, and node results persist across runs.

Flag	Meaning
<code>--once <file.pp></code>	Run once and exit (the default).
<code>--watch <file.pp></code>	Run, then re-run when observed cells change.
<code>--reconcile <root> <file.pp></code>	Treat the program's final value as a desired file tree and converge <code><root></code> to it.

12.3 Capabilities and grants

Side effects that touch the world require a capability, and capabilities enter a run only here, on the command line. `--grant` is repeatable; each grant is a colon-separated spec. Paths are canonicalized at the grant, so downstream authority checks and cell identities see one spelling.

Grant spec	Authority granted
<code>fs:<path>:ro</code>	Read access to a filesystem path (and everything under it).
<code>fs:<path>:rw</code>	Read and write access to a path.
<code>fs:<path>:wo</code>	Write-only access to a path.
<code>net:<host></code>	Network access to a host, any port.
<code>net:<host>:<port></code>	Network access to a host on one port.
<code>secret:<path></code>	Read a path as a sealed value (bytes never enter the store; <code>unseal</code> is the one way out).
<code>process</code>	Spawn, signal, and reap child processes.

Usage: `pp --grant fs:/tmp/build:rw --grant process <file.pp>`. Any of the world-touching primitives (`slurp`, the `fs/proc` domains, network effects) raises a capability error if the matching grant is absent.

12.4 The store and auditing

Every node's result is content-addressed and cached in `~/.pp/store`. These flags inspect, bypass, audit, and reclaim that store.

Flag	Meaning
<code>why <file.pp> (--why)</code>	Run the file and explain each node's cache hit or miss. Capability-filtered: you only see nodes you had authority to observe.
<code>--no-cache <file.pp></code>	Skip cache reads and recompute every node; results are still written to the store.
<code>--check <file.pp></code>	Determinism audit: run each node twice and flag any whose result differs (a volatile node). With a non-serial <code>--schedule</code> , also re-runs the whole program serially and compares the desired-state hash. Exits 1 if anything is flagged.
<code>graph</code>	Print the cell→node dependency graph reconstructed from stored traces. Needs no file.
<code>gc</code>	Explicit store garbage collection (never automatic). Walks the last N wanted graphs through durable child/result/blob edges, then sweeps unreachable data.
<code>gc --gc-keep-epochs <N></code>	Keep the last N epochs (a small built-in default; $N > 0$).
<code>gc --gc-grace-seconds <S></code>	Spare anything younger than S seconds from the sweep ($S \geq 0$).

12.5 Reconcile, watch, and supervise

These turn a program's value into managed state in the world. `--reconcile` materializes a `{relpath → content}` map as a file tree; `--supervise` drives a `{name → spec}` map of long-running processes. Both use their built-in domain and both are effectful, so both need grants.

Flag	Meaning
<code>--reconcile <root> <file.pp></code>	Materialize the program's map value as a file tree under <code><root></code> , creating/updating/deleting by content hash. Uses the built-in <code>fs</code> domain. Needs <code>fs:<root>:wo</code> (or <code>:rw</code>).

Flag	Meaning
<code>--supervise <file.pp></code>	Converge the program's process map: start/restart/stop services to match it. Uses the built-in process domain. Needs process. Pair with <code>--watch</code> to keep services alive.
<code>--watch <file.pp></code>	Run, then poll the observed cells and re-evaluate whenever one changes. Combines with <code>--reconcile/--supervise</code> to re-converge on drift.
<code>--stabilize</code>	With <code>--watch</code> : propagate changes by dirtying only the affected nodes (push stabilization) rather than re-running cold.
<code>--watch-interval <s></code>	Poll interval for <code>--watch</code> , in seconds (default 1.0).
<code>--fenced-policy retry abort ask</code>	How to treat a fenced (non-convergent) action left in an unknown state by a prior crash. Default abort.

12.6 Islands

Islands are content-addressed module pins — a git ref resolved to a hash and recorded inline in the source. These flags manage the pins.

Flag	Meaning
<code>island-pins <file.pp></code>	List the file's island forms with their pin and cache status. Does not run the program.
<code>--update <file.pp></code>	Re-resolve each island ref and rewrite the inline pins in place, then run. Implies <code>--fetch-islands</code> .
<code>--fetch-islands</code>	Allow a git fetch for island pins not already cached (off by default — an offline run only uses what the store already holds).

12.7 Scheduling and distribution

By default, pp computes node misses serially, in-process. `--schedule` changes where and how they run, up to placing them on other cluster members. The M5 cluster flags establish trust (`cluster-init`, `--mint-token`) and move desired state between machines by hash (`--publish-object`, `--desired-object`).

Flag	Meaning
<code>--schedule serial</code>	Compute node misses one at a time, in-process (the default).
<code>--schedule parallel:<N></code>	Fan out independent misses across up to N workers.
<code>--schedule race:<N></code>	Race up to N redundant computations, taking the first to finish.
<code>--schedule remote:<member></code>	Place misses on a named cluster member.
<code>cluster-init</code>	Mint <code>~/.pp/cluster/{secret,id}</code> — the cluster's trust anchor.
<code>--mint-token <out> <ttl-secs></code>	Mint a signed cluster token into <code><out></code> , carrying whatever <code>--grant</code> specs accompany it, valid for <code><ttl-secs></code> .
<code>--member-name <n> <file.pp></code>	Host-qualified distribution: treat the desired state as a <code>{host → {domain → desired}}</code> map and

Flag	Meaning
	converge only host <n>'s slice. Combine with --reconcile/--supervise.
--publish-object <shared-root> <file.pp>	Run the program, store its fully-forced value and canonical tree blobs in a shared local-dir store by hash, and print the hash.
--desired-object <hash> <shared-root>	Pull a published value by <hash> from <shared-root> and converge it directly — never runs a program to derive it. Takes --member-name and the reconcile/supervise flags.

12.7.1 Internal transport (not for hand use)

pp invokes these on itself to move artifacts and dispatch remote work. They are low-level and explicit by design; a normal run never types them.

Flag	Meaning
--transport-push object blob trace <id> <root>	Copy one artifact into a shared local-dir store.
--transport-pull object blob trace <id> <root>	Copy one artifact out of a shared local-dir store (re-hash-verified on ingest).
--serve-hit <key> <token-file> <shared-root> <reply-file>	Capability-gated cache lookup served to a dispatcher.
--recv-hit <reply-file> <shared-root>	Ingest a serve-hit reply.
--remote-node <token> <pins> <root> <keys> <reply>	The cluster-member side of remote placement; authority comes from the verified token, not --grant.

12.8 Observation pinning

These capture and replay the exact set of world observations a run made — the seam that lets one machine pin another's reads (and the basis of remote placement's pin wire).

Flag	Meaning
--pin-file <path> <file.pp>	Preseed the run's cell observations and probe values from a file of (pin ...) / (pin-probe ...) lines before the program runs, so it never observes its own disk for a pinned cell.
--dump-pins <path> <file.pp>	After the run, write every observed cell and probe value to <path> as (pin ...) / (pin-probe ...) lines. A probe value that is not plain data (code, a handle, a sealed secret) is skipped with a warning.

The pin-file's (pin ...) / (pin-probe ...) lines are a small wire format, not pp source. They are not read by either pp reader, so they keep their fixed parenthesized shape regardless of the surface a program is written in.

13 Builtins and the standard library

The two layers of pp’s vocabulary that are not special forms: builtins from the saved image (`lisp/runtime/language.lisp`) and the pp-written standard library (`stdlib/*.pp`). Special forms belong to the language reference.

Signatures use pp calling syntax: `name(arg, ...)`. A trailing `...` marks a variadic position; `[arg]` marks an optional one. Unless noted, a builtin forces the arguments it inspects; `cons`, `list`, `hash-map`, and `map` are deliberately lazy in the values they carry.

13.1 Dune adapter

`load("stdlib/dune.pp")` provides `dune-build(adapter, spec)`. `:working-tree` observes a development tree and returns selected Dune output as a canonical artifact tree. `:closed-source` executes the equivalent immutable source/tool request through `run-closed!`. The library also exposes `dune-closed-request(spec)` so release requests can be inspected, hashed, and transported without execution. Dune target and output policy lives entirely in this library.

13.2 Core builtins

These are always in scope — no `load` needed.

13.2.1 Arithmetic and comparison

Signature	Description
<code>+(n, ...)</code>	Variadic sum; identity 0. Mixing ints and floats promotes to float.
<code>-(a, b)</code>	Subtraction of exactly two numbers.
<code>*(n, ...)</code>	Variadic product; identity 1.
<code>/(a, b)</code>	Division of two numbers; the divisor must be non-zero.
<code>mod(a, b)</code>	Integer remainder; the divisor must be non-zero.
<code>=(a, ...)</code>	Structural equality; true iff all arguments are equal.
<code><(a, ...), >(a, ...)</code>	Variadic chained ordering; true iff each adjacent pair is strictly ordered.
<code><=(a, ...), >=(a, ...)</code>	As above, non-strict.

13.2.2 Lists and pairs

Signature	Description
<code>cons(a, b)</code>	Build a pair, storing both parts unforced (lazy).
<code>car(p)</code>	First element of a pair; <code>nil</code> for <code>nil</code> .
<code>cdr(p)</code>	Rest of a pair; <code>nil</code> for <code>nil</code> .
<code>list(x, ...)</code>	Build a proper list from its arguments (lazy in the elements).
<code>apply(f, ...segments)</code>	Apply <code>f</code> to the concatenated argument lists; used by call-spread syntax.
<code>map(f, lst)</code>	Apply <code>f</code> to each element, consing the results without forcing them — the parallel fan-out point the scheduler batches on.
<code>nil?(x)</code>	True if <code>x</code> is <code>nil</code> .

13.2.3 Vectors, maps, and sets

Signature	Description
<code>vector(x, ...)</code>	Build a vector (lazy in the elements).
<code>vector-get(v, i)</code>	Element <code>i</code> of vector <code>v</code> ; out-of-bounds is an error.
<code>hash-map(k, v, ...)</code>	Build a map from alternating keys and values; keys are forced, values stay lazy.
<code>hash-map-get(m, k)</code>	Value bound to <code>k</code> in <code>m</code> , or <code>nil</code> .

Signature	Description
<code>map-insert(m, k, v)</code>	A new map with <code>k</code> bound to <code>v</code> (replacing any existing binding).
<code>map-remove(m, k)</code>	A new map without key <code>k</code> .
<code>map-keys(m)</code>	The keys of <code>m</code> as a list.
<code>map-valz(m)</code>	The values of <code>m</code> as a list.
<code>hash-set(x, ...)</code>	Build a set (lazy in the elements).

13.2.4 Type predicates

Each takes one argument, forces it, and returns a boolean.

Predicate	True when the value is...
<code>int?(x)</code> , <code>float?(x)</code>	an integer / a float.
<code>string?(x)</code> , <code>bool?(x)</code>	a string / a boolean.
<code>keyword?(x)</code> , <code>symbol?(x)</code>	a keyword / a symbol.
<code>pair?(x)</code>	a pair or <code>nil</code> .
<code>vector?(x)</code> , <code>map?(x)</code> , <code>set?(x)</code>	a vector / a map / a set.
<code>fn?(x)</code>	a closure or builtin.
<code>thunk?(x)</code>	an unforced thunk (does not force its argument).
<code>capability?(x)</code>	a capability.

13.2.5 Strings and numbers

Signature	Description
<code>string-append(s, ...)</code>	Concatenate; non-string arguments are rendered first.
<code>string-length(s)</code>	Length of a string in bytes.
<code>->string(v)</code>	Render any value without string quotes; strings are returned unchanged.
<code>string-split(s, sep)</code>	Split <code>s</code> on the single-character <code>sep</code> , dropping empty fields.
<code>string-index(s, sub)</code>	Index of the first occurrence of <code>sub</code> , or <code>nil</code> .
<code>string-trim(s)</code>	<code>s</code> with leading and trailing whitespace removed.
<code>string-sub(s, start, len)</code>	The <code>len</code> -byte substring at <code>start</code> ; out-of-bounds is an error.
<code>number->string(n)</code>	Decimal rendering of an int or float.
<code>string->number(s)</code>	Parse to an int, else a float, else <code>nil</code> .

13.2.6 I/O and control

Signature	Description
<code>print(x, ...)</code>	Deep-force each argument, print them concatenated with a trailing newline, and return <code>nil</code> .
<code>not(x)</code>	Logical negation; <code>nil</code> counts as false.
<code>error(msg)</code>	Raise an error with string message <code>msg</code> .
<code>exit([n])</code>	Terminate the run with status <code>n</code> (default 0).
<code>slurp(path)</code>	Read a file to a string. Needs an <code>fs</code> read grant (or a <code>secret</code> grant, which yields a sealed value); capability-free inside a node's scratch sandbox.
<code>argv()</code>	The program arguments after <code>--</code> , as a list of strings. Recorded as an <code>argv</code> : observation.
<code>env-get(name)</code>	The environment variable name, or <code>nil</code> . Recorded as an <code>env</code> : observation.

13.2.7 Filesystem observation

Capability-gated presence checks, recorded as `stat`: cells so a node recomputes exactly when the path appears, disappears, or changes kind — never reading contents.

Signature	Description
<code>file-exists?(path)</code>	True if anything exists at path. Needs an <code>fs read</code> grant.
<code>dir?(path)</code>	True if path is a directory. Needs an <code>fs read</code> grant.

13.2.8 Hashing and the content store

Signature	Description
<code>hash-value(v)</code>	A canonical structural content hash of any value — order-independent for maps and sets.
<code>hash-string(s)</code>	The SHA-256 hex digest of a string's raw bytes (pure; no store I/O).
<code>blob(s)</code>	Ingest bytes into the content store, returning the SHA-256 identity.
<code>blob-get(hash)</code>	The inverse of <code>blob</code> : the stored bytes for a blob identity.

13.2.9 Capabilities

Signature	Description
<code>current-capabilities()</code>	Reify the ambient capability set as of the call — an observation of the ceiling, never a mint.
<code>cap-compose(c, ...)</code>	Compose several capabilities into one.
<code>cap-restrict(cap, scope, [:ro :rw :wo])</code>	Narrow cap to a scope string and, optionally, a weaker mode.
<code>cap-none()</code>	The empty capability (grants nothing).

13.2.10 Effects, domains, and metaprogramming

`stdlib/runtime.pp` provides `schedule-serial`, `schedule-parallel`, `schedule-race`, `schedule-custom`, `runtime-manifest`, reporter constructors, and policy helpers. `configure-runtime` installs a manifest at script scope. `stdlib/domain.pp` provides generic domain and probe registration helpers.

Signature	Description
<code>register-domain({...})</code>	Register a write-domain from a spec map (<code>:name</code> , <code>:namespace</code> , <code>:owner</code> , <code>:cap</code>). Script-tier only — not inside a node body.
<code>register-probe(name, observe-fn, read-cap)</code>	Register a read-only probe: a domain with no write authority. Script-tier only.
<code>probe(name)</code>	Read a registered probe's value, pinned once per pass and recorded as a <code>probe: cell</code> .
<code>fenced(kind, spec)</code>	Register a non-convergent (fenced) action for the reconciler to drain after convergent state is applied.
<code>collect(results)</code>	Partition <code>[:ok, value]</code> and <code>[:err, error]</code> results into one aggregate result.
<code>unseal(v)</code>	Convert a sealed value to a string — the one sanctioned exit from <code>secret: bytes</code> .
<code>eval-pp(code)</code>	Parse and evaluate a code string in the current run's environment and macro table.

Signature	Description
<code>apply-pp(fn, args)</code>	Apply <code>fn</code> to a list of already-evaluated arguments.
<code>force-deep(v)</code>	Fully force a value and everything reachable from it.
<code>read-string(src)</code>	Parse a source string to a quoted value (or a vector of them).
<code>gensym([prefix])</code>	A fresh, unwritable symbol for hygienic macro bindings.

The reader forms `quasiquote`, `unquote`, and `unquote-splicing` are also registered as builtins, normally reached through the reader's `quasiquote` syntax; outside a `quasiquote` the latter two are an error.

13.3 Lists

`stdlib/list.pp` — load with `load("list.pp")`. Note that `map` is not here: it is a builtin, on purpose, so a loaded list library cannot shadow the scheduler's fan-out point.

Signature	Description
<code>filter(pred, lst)</code>	A lazy list of the elements satisfying <code>pred</code> .
<code>foldl(f, acc, lst)</code>	Left fold, strict in the accumulator.
<code>foldr(f, acc, lst)</code>	Right fold, lazy.
<code>range(start, end)</code>	The integers from <code>start</code> (inclusive) to <code>end</code> (exclusive).
<code>take(n, lst)</code>	The first <code>n</code> elements.
<code>drop(n, lst)</code>	<code>lst</code> without its first <code>n</code> elements.
<code>nth(n, lst)</code>	Zero-based element access; <code>nil</code> past the end.
<code>length(lst)</code>	Element count (strict — forces the whole list).
<code>each(f, lst)</code>	Apply <code>f</code> to each element for effect; returns <code>nil</code> .
<code>append(a, b)</code>	Concatenate two lists (lazy in <code>b</code>).
<code>reverse(lst)</code>	Strict reversal.
<code>member?(x, lst)</code>	Structural membership test.

13.4 Maps

`stdlib/map.pp` — load with `load("map.pp")`; depends on `list.pp`.

Signature	Description
<code>map-has?(m, k)</code>	Whether <code>k</code> is a key of <code>m</code> .
<code>map-merge(a, b)</code>	<code>a</code> with every binding of <code>b</code> inserted; <code>b</code> wins on collision.

13.5 Strings

`stdlib/string.pp` — load with `load("string.pp")`; no dependencies.

Signature	Description
<code>string-join(sep, lst)</code>	Join a list of strings with <code>sep</code> between elements.
<code>starts-with?(s, prefix)</code>	Whether <code>s</code> begins with <code>prefix</code> .
<code>ends-with?(s, suffix)</code>	Whether <code>s</code> ends with <code>suffix</code> .
<code>lines(s)</code>	Split <code>s</code> into lines, dropping empty fields.

13.6 Domains

The built-in `--reconcile/--supervise` domains register their policy natively. `stdlib/domain-fs.pp` and `stdlib/domain-proc.pp` package the same policies as ordinary pp libraries for explicit registration through `register-fs-domain(root, write-cap)` and `register-proc-domain(write-cap)`. The trusted mechanics they call — `tree-observe`, `materialize-file`, `proc-spawn`, and so on — are runtime providers reached only through `perform`. You normally interact with a domain through the registration entry point; the rest is its internal policy, listed here for readers of the source.

The one entry point you call directly:

Signature	Description
<code>register-fs-domain(root, write-cap)</code>	Register the filesystem domain rooted at <code>root</code> , converging a canonical tree value.
<code>register-proc-domain(write-cap)</code>	Register the process domain, converging a <code>{name → spec}</code> map (spec: <code>cmd/args/env/cwd</code>).

13.6.1 Filesystem policy internals (`domain-fs.pp`)

Signature	Description
<code>fs-content-hash(c)</code>	A file descriptor's blob identity.
<code>fs-content-bytes(c)</code>	The bytes named by a file descriptor.
<code>fs-validate-rel-part(rel, part)</code>	Reject <code>..</code> in a desired path component.
<code>fs-validate-rel(rel)</code>	Validate that a desired path is relative and traversal-free.
<code>fs-plan-item(kind, rel, content)</code>	Build one create/update/delete plan item.
<code>fs-diff-for(root)</code>	Return the pure (observed desired) → plan diff, closed over <code>root</code> .
<code>fs-apply-item(root, item)</code>	Materialize or remove one file per plan item.
<code>fs-apply-for(root)</code>	Return the apply function that runs each plan item.

13.6.2 Process policy internals (`domain-proc.pp`)

Signature	Description
<code>proc-state-key(name)</code>	The domain-state key under which a service's record is stored.
<code>proc-known-names()</code>	The domain's own index of tracked service names.
<code>proc-remember!(name, pid, spec, known)</code>	Record a running service and add it to the known set.
<code>proc-forget!(name, known)</code>	Clear a service's record and drop it from the known set.
<code>proc-observe-one(name)</code>	Observe one service: its spec if alive, <code>:stopped</code> if tracked-but-dead, else <code>nil</code> .
<code>proc-observe()</code>	Reap zombies, then observe every tracked service.
<code>proc-plan-item(kind, name, spec)</code>	Build one start/restart/stop plan item.
<code>proc-spec-eq?(a, b)</code>	Compare two specs by canonical hash-value (immune to map-order differences).
<code>proc-diff(observed, desired)</code>	The pure diff: start / restart / stop by spec change.
<code>proc-stop-current!(name)</code>	Stop a service at its currently-recorded pid.
<code>proc-apply-item(known, item)</code>	Apply one plan item, returning the updated known set.

Signature	Description
proc-apply(plan)	Fold every plan item, then persist the known set once.